Développement Fullstack

Rapport final d'évaluation

Nom et prénom :

Mansouri Mohammed

Filière :

4A INFO FISA

Date de rédaction :

25/03/2026

Année scolaire :

2025 - 2026

1. Description du système : **1**
1.1 Contexte et objectifs du projet : 1
1.2 Architecture technique et choix technologiques 1
1.3 Fonctionnalités, expérience utilisateur et perspectives 2
2. Human-AI Co-Development **3**
2.1 Approche adoptée : Spec-Driven Development 3
2.2 Contribution de l'IA et outils utilisés 3
2.4 Analyse critique de la collaboration 3
2.4.1 Limites de l'IA 3
2.4.2 Apport global de l'IA 4
3. Project Management **4**
3.1 Organisation du projet 4
3.2 Gestion du versioning et méthodologie de développement 5
3.4 Qualité du projet 6

1. Description du système :

1.1 Contexte et objectifs du projet :

Le projet **EventFire** s'inscrit dans le cadre du module de programmation web fullstack. Il vise à concevoir une application web complète permettant la gestion d'événements musicaux et culturels. L'objectif principal est de centraliser les informations relatives aux événements et de proposer une plateforme accessible à différents types d'utilisateurs.

D'une part, les visiteurs non authentifiés peuvent consulter les événements disponibles, effectuer des recherches et accéder aux détails complets. D'autre part, les administrateurs disposent d'un espace sécurisé leur permettant de gérer les événements via un tableau de bord dédié.

Ainsi, le projet répond à un besoin concret de gestion et de visualisation de données événementielles dans un environnement web moderne.

1.2 Architecture technique et choix technologiques

Le système repose sur une architecture MERN (MongoDB, Express, React, Node.js), permettant une séparation claire entre les différentes couches. Le frontend est développé avec React et Vite pour une interface dynamique, tandis que le backend utilise Node.js et Express pour exposer une API REST. La base de données MongoDB est déployée via Docker, garantissant un environnement reproductible.

Afin d'assurer la robustesse du système, plusieurs choix techniques ont été adoptés : l'utilisation de JWT pour l'authentification sécurisée, Mongoose pour la

modélisation des données, ainsi que l'architecture MVC côté backend pour une meilleure organisation du code. Ces choix permettent de concevoir une application maintenable, sécurisée et évolutive.

1.3 Fonctionnalités, expérience utilisateur et perspectives

Le projet a été développé de manière progressive. La première version inclut les fonctionnalités essentielles, notamment la consultation publique des événements, l'affichage des détails (titre, date, lieu, image), l'authentification administrateur via JWT ainsi qu'un dashboard permettant la gestion complète des événements (CRUD). Cette base constitue le socle fonctionnel du système.

Une seconde phase a permis d'intégrer des fonctionnalités avancées telles que la recherche par mot-clé, le filtrage par catégorie et un dashboard statistique affichant des indicateurs comme les revenus, le nombre de tickets vendus et le taux d'occupation. Ces améliorations renforcent la valeur fonctionnelle de l'application.

Par ailleurs, des extensions supplémentaires ont été développées au-delà des exigences initiales, notamment un système de réservation utilisateur, l'authentification via Google OAuth, l'upload d'images et la mise en place de mécanismes de sécurité avancés. Ces ajouts témoignent d'une volonté d'approfondir le projet.

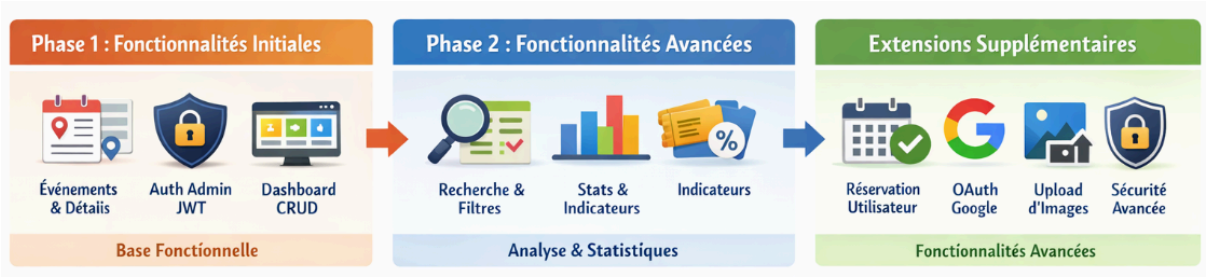

figure 1 : Schéma des fonctionnalités implémentés progressivement

L'expérience utilisateur a été pensée pour être simple et fluide, avec une interface React dynamique permettant une navigation rapide. Le filtrage en temps réel, les indicateurs visuels et la gestion des événements passés améliorent l'interaction globale, tandis que le dashboard admin facilite la gestion des données.

Le résultat final est une application fullstack complète et fonctionnelle, intégrant un frontend connecté au backend, une API sécurisée et une base de données opérationnelle. Néanmoins, certaines perspectives d'amélioration restent envisageables, notamment le déploiement en environnement cloud, l'optimisation des performances et l'ajout de fonctionnalités en temps réel.

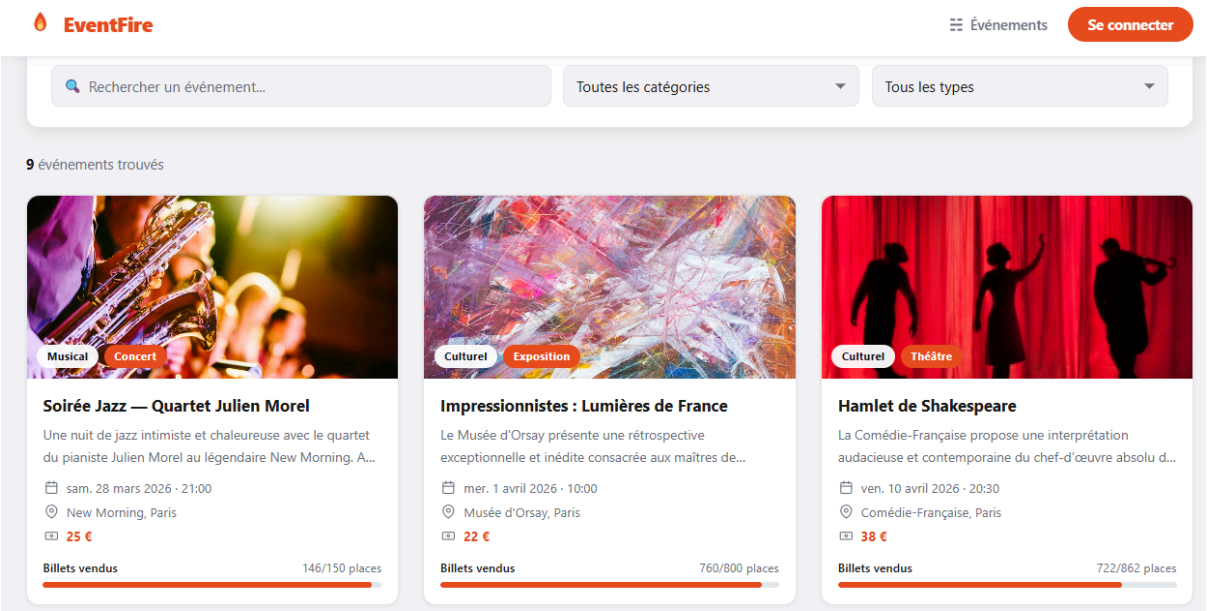

figure 2 : Interface des événements lors de la visite public

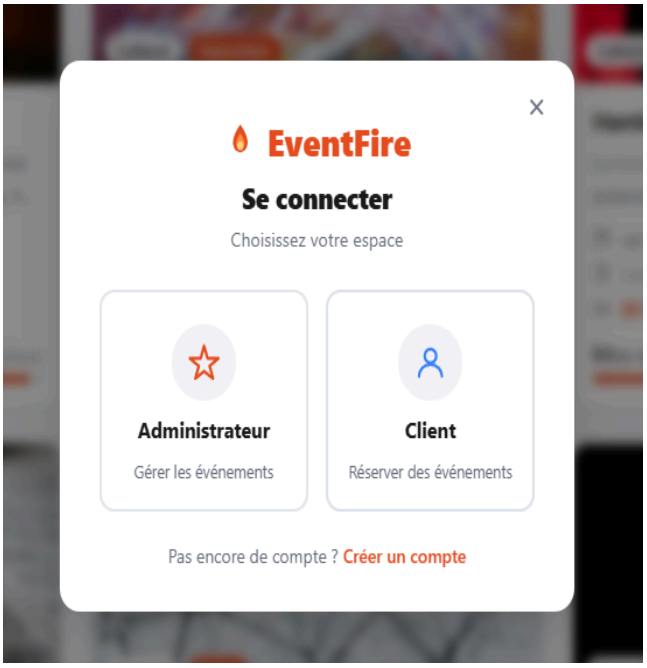

Figure 3 : Interface pour se connecter sur les différents espaces d'app web

2. Human-AI Co-Development

2.1 Approche adoptée : Spec-Driven Development

Le développement du projet repose sur une approche Spec-Driven Development, dans laquelle chaque fonctionnalité est définie avant son implémentation.

Cette méthodologie s'appuie sur :

- Le contrat de projet
- La définition des releases dans le contrat projet
- Les maquettes initiales Figma
- Claude.md

Elle permet de structurer le développement et de garantir la cohérence du système.

2.2 Contribution de l'IA et outils utilisés

L'intelligence artificielle a été utilisée comme un assistant de développement tout au long du projet. Elle a notamment contribué à la génération de composants React, à la création de routes et middlewares Express, à la modélisation des données ainsi qu'à l'explication de concepts techniques et à la rédaction de documentation. Son utilisation a permis de gagner du temps, en particulier sur des tâches répétitives ou standardisées.

L'outil principal mobilisé est *Claude (Anthropic)*, utilisé dans différents contextes tels que la génération de code, les suggestions d'architecture et l'assistance à la compréhension de notions complexes. L'utilisation de l'IA s'est faite de manière itérative, avec une validation humaine systématique afin de garantir la qualité, la cohérence et l'adéquation des solutions proposées.

2.4 Analyse critique de la collaboration

2.4.1 Limites de l'IA

L'utilisation de l'intelligence artificielle dans le projet a mis en évidence plusieurs limites concrètes.

Tout d'abord, l'IA ne dispose pas d'une vision réelle du rendu final. Par exemple, elle a généré du code correct en apparence, mais l'affichage était incorrect (éléments inversés ou mal positionnés). Ce type de problème n'est détectable qu'après exécution dans le navigateur, ce qui nécessite une correction manuelle du CSS.

Figure 4 : Exemple concret génération d'erreur code CSS

Certaines propositions de l'IA se sont révélées trop génériques et inadaptées au contexte, comme l'ajout de Google OAuth pour un compte administrateur sans prise en compte de la sécurité, ce qui a nécessité une correction. De plus, l'IA ne détecte pas les problèmes liés à l'environnement technique : elle proposait des corrections de code alors que l'erreur venait de MongoDB ou Docker, ce qui a demandé une intervention humaine.

2.4.2 Apport global de l'IA

L'intelligence artificielle a constitué un levier de productivité important dans le projet. Elle a permis d'accélérer le développement en générant rapidement du code et en facilitant l'implémentation de certaines fonctionnalités.

Elle a également apporté un réel soutien dans la compréhension de concepts techniques, permettant de débloquer rapidement des points complexes et d'avancer plus efficacement.

Enfin, l'IA a servi d'outil d'assistance global, en proposant des solutions, en structurant certaines parties du code et en accompagnant la rédaction, tout en laissant au développeur la responsabilité des choix et des validations.

3. Project Management

3.1 Organisation du projet

Le projet a été structuré de manière progressive en plusieurs phases : une première release dédiée aux fonctionnalités de base, suivie d'une seconde intégrant des fonctionnalités avancées, puis des extensions (sécurité, UX, réservation).

Pour organiser le travail, j'ai utilisé des outils de gestion de projet comme *Linear* au début, puis *GitHub Projects*, en définissant des issues et des milestones. Chaque tâche était associée à une issue, traitée étape par étape.

Le développement a été mené de manière proche d'une logique de sprints : une fois une tâche terminée, elle était validée puis fermée, avant de passer à la suivante. Cette approche m'a permis de garder une vision claire de l'avancement du projet et de ce qu'il restait à réaliser (pilotage du projet).

Figure 5 : Exemple issues créer lors du projet

3.2 Gestion du versioning et méthodologie de développement

Le versioning a été réalisé avec GitHub, en utilisant des commits sémantiques (feat, fix, docs) pour structurer les modifications.

Le développement s'est fait par issues, avec des versions intermédiaires pour chaque fonctionnalité. Une fois les tâches finalisées, un merge global a été effectué.

En revanche, seules les branches main et V2 ont été conservées, garantissant un projet propre et stable.

à la fin, la méthodologie de développement que j'ai utilisé dans ce projet est la suivante (comme montre le schéma) :

Cette méthode m'a permis d'avancer progressivement, tout en améliorant la qualité du système à chaque étape.

3.4 Qualité du projet

Le projet se distingue par une bonne qualité globale :

- Structure du code claire, facilitant la maintenance et l'évolution
- Architecture cohérente, avec une séparation nette des responsabilités
- Documentation suffisante, assurant la compréhension du système
- Fonctionnalités enrichies, allant au-delà du socle initial

Des améliorations ciblées ont également été apportées, notamment en sécurité, en expérience utilisateur et en gestion des fonctionnalités, renforçant ainsi la robustesse et la pertinence du projet.